

과제 2 — AI 활용 산출물 및 과정

무엇을 만들었나

"투자 아이디어 → 코드 → 백테스트" **파이프라인(Research-to-Backtest)**. 사람이 세운 투자 논지(과제 1의 네이버 "실적 확인형" 같은)를 ① DART·KRX 데이터 수집·정규화 ② 전략 규칙(DSL) ③ Point-in-Time 백테스트 ④ 근거·리포트 자동화로 잇는 시스템이다. 펀드매니저의 아이디어를 Python 퀀트 모델로 구현·검증하는 업무를 그대로 도구화한 것이다.

- 산출물: GitHub — <https://github.com/yachom/research-to-backtest> (파이썬 45k 라인, 테스트 837건, 실데이터 end-to-end 실행, 실행 가이드 README).
- 실행 예(네이버): 과제 1의 "실적 확인형" 논지를 전략 DSL로 변환해 2016~2026 백테스트·15섹션 보고서까지 관통했다(부록). 결과는 누적 -17.9%로, **기계적 재무 프록시가 네이버의 정성 촉매(AI 인프라 전환)를 포착하지 못함**을 보여준다 — 이 실패가 로드맵의 근거다.

어떤 도구·프롬프트로

두 층위로 AI를 썼다.

(1) 만들 때 — **Claude Code 멀티에이전트 오케스트레이션**. 메인 세션(설계·명세·병합·검증)이 구현을 **워크트리 격리 병렬 에이전트**에 위임했다. 여기서 **프롬프트 = 명세 파일**이다 — docs/specs/*_md 에 파일 소유권과 인터페이스 계약을 먼저 고정해 병합 충돌을 설계 단계에서 차단했다. 4개 웨이브·12개 병렬 트랙에서 병합 충돌은 1건이었다.

명세 프롬프트 예시(발췌): "너는 T1 구현 에이전트다. \$1 파일 소유권 박 수정 금지. build_financial_datasets를 조립해 CLI를 붙이고, 실데이터 스모크로 136 metrics 재산출을 확인하라. 명세와 실측이 다르면 우회하지 말고 사유와 함께 보고하라."

(2) 시스템 안에서 — **Claude(Haiku)**. 파이프라인 내부에서 LLM은 **후보 정리·초안만** 맡는다(분석 후보·가설 후보·전략 DSL 초안·결과 설명). 모델은 configs/llm.yaml 에 고정, 프롬프트는 */prompts/*_v1.txt 로 버전 관리, 모든 호출은 ai_usage_log.jsonl 에 기록된다. 인증은 API 종량과금을 피하려고 **개인 구독 계정의 OAuth 토큰** (Claude Agent SDK)을 쓴다 — run당 호출이 소수라 이 방식으로 충분하다.

어떻게 풀었나

- 역할 분리를 코드로 강제**: AI는 후보·초안, 사람은 관점·가설·승인·해석. 승인 게이트 (I2-상태 머신)가 미승인 단계 실행을 예외로 막는다 — "AI가 판단한 것처럼 보이는" 구조를 회피했다(과제 1의 "본인의 시각" 요건과 직결).
- 데이터 신뢰성 우선**: 모든 재무값에 공식 접수 다음 거래일(available_from)을 부여하고 as-of 결합만 허용해 미래 정보 유입을 차단(Point-in-Time). API 수치와 XBRL 원본을 교차검증(연간 100% 일치)한 뒤에만 AI에 근거로 넘긴다.
- AI 산출물을 믿지 않고 검증**: LLM이 인용한 근거 ID가 실제 데이터에 없으면 자동 재요청, 전략 초안은 컴파일러로 재검증, 결과는 사람이 해석·판정한다.
- 한계 명시**: 하이퍼콜라바X 단독 손익처럼 공개되지 않은 값은 추정하지 않고, 백테스트가 검증하는 것은 논지의 "재무 확인" 대리 명제까지임을 밝혔다.

검증 가능성

- 개발 과정: git 커밋 이력·Co-Authored-By 트레일러·docs/PROGRESS.md (웨이브별 스냅샷).
- 시스템 내 AI 사용: outputs/{run_id}/ai_usage_log.jsonl + 프롬프트 버전 파일.
- 동작: make check (테스트·타입·린트) 및 실데이터 실행 로그.

우대사항 연결 — 충족 범위와 한계

- 충족**: 공사·거래소 데이터 API(OpenDART·KRX) 연동으로 금융 데이터 크롤링·수집·가공, 백테스트(**시물레이션**) 플랫폼 산출물, AI로 Python 기술 허들을 스스로 넘어 구현·검증한 사례. 데이터 수집 계층은 어댑터(MarketDataSource 인터페이스)로 추상화해 다른 증권사 데이터 소스로 교체 가능하도록 설계했다.
- 미충족**: 실제 **증권사(브로커리지) 주문 API를 통한 시스템 트레이딩(라이브 주문 집행)**은 이번 산출물에 포함하지 않았다. 현재 플랫폼은 신호 생성 → 체결 시물레이션 → 성과·강건성 검증까지를 담당하며, **주문 집행 계층은 미구현**이다(아래 로드맵 4).

"시스템 트레이딩"을 데이터 수집(주요업무)과 실주문 집행(우대사항)으로 구분했다 — 이번 제출물은 전자를 실연하고 후자는 로드맵으로 남긴다.

후후 보강 방안 (로드맵)

현 시스템은 **정형 데이터**(가격·수급·XBRL 재무)를 근거로 하고, 사용자가 구조화된 형태(관점·가설·전략 DSL)로 논지를 입력한다. 아직 **자연어 아이디어만으로 전략을 구성하기**에는 근거 데이터와 판단 계층이 부족하다. 다음 순서로 확장한다.

- 비정형 데이터 수집**: 뉴스·공시 원문·산업 리포트를 수집·요약해 정성 촉매(예: 과제 1의 국책 GPU·국방 AX TF)를 시스템이 직접 근거화한다. 여기서 위 OAuth 방식의 한계가 드러난다 — 다수 기사 요약은 호출·출력량이 커 구독 토큰의 사용량 한도·장출력 타임아웃에 걸리고, 고빈도 자동 처리는 구독 약관상 회색지대다. 뉴스 요약 규모로 가려면 종량과금 API 키가 필요해, OAuth로 피했던 **비용 문제가 다시 등장**한다 — 이 비용↔확장 트레이드오프가 이 단계의 핵심 설계 결정이다.
- 에이전트 판단 계층**: 정성+정량 근거를 종합해 에이전트가 전략 후보를 제시한다(현재는 사람이 DSL로 변환 — 확장 시 논지에서 전략 초안까지 자동).
- 자연어 지시 인터페이스**: "실적이 확인되면 진입하되 CAPEX가 급증하면 보수적으로" 같은 자연어 지시를 전략 DSL로 자동 번역한다(현재는 구조화 입력 필요).
- 실주문 집행**: 백테스트로 검증된 전략을 페이퍼트레이딩 → 증권사 주문 API 실행으로 연결한다(토스증권 키 발급 완료, 데이터 어댑터와 동일 패턴).

목표는 "펀드매니저의 자연어 아이디어 → 근거 수집·에이전트 전략 제시 → 백테스트 → 실행" 전 과정의 자동화이며, 이번 산출물은 그중 **정형 데이터 기반 구현·검증까지**를 완성했다.